

## 实验九 UML、逻辑与软件体系结构设计报告

项目名称：BlogHub 博客社区平台

小组成员：王佳闻、王榕祺、杨小英、田依然

完成日期：2026-05-22

### 第一部分：UML 建模语言学习总结

#### 1.1 UML 概述

统一建模语言（Unified Modeling Language, UML）是面向对象系统分析和设计的标准化图形建模语言，由 Grady Booch、James Rumbaugh 和 Ivar Jacobson 融合各自方法学于 1997 年提出，现由 OMG 维护。UML 2.5 规范定义了 14 种图型，分为结构图（类图、组件图、部署图等）和行为图（用例图、状态机图、活动图、序列图等）。UML 的核心价值在于提供一种跨角色（用户、分析师、设计师、开发人员）沟通的通用视觉语言，使系统复杂度得以在多个抽象层次上表达。

#### 1.2 UML 在 BlogHub 项目中的应用

在本项目中，我们使用 UML 完成了以下建模工作：

| 图类<br>型 | 用途                                                 | 对应项目产出                                        |
|---------|----------------------------------------------------|-----------------------------------------------|
| 用例图     | 描述参与者（访客、用户、版主、管理员）与系统功能的交互关系                      | 权限矩阵、功能需求 FR-01~FR-27                         |
| 类图      | 定义核心实体（User, Post, Section, Comment 等）及其属性、方法、关联关系 | 数据库表结构、ORM 模型                                 |
| 状态机图    | 展示文章、板块申请、举报等实体的生命周期状态转换                           | 状态图（Post, SectionApplication, Report）及 OCL 约束 |
| 活动图     | 建模业务流程（如板块申请审批、文章发布审核）                             | 泳道图、DFD 流程描述                                  |
| 序列图     | 表示对象间消息交互的时间顺序（如用户发表评论的调用链）                        | API 接口设计文档（隐含）                                |
| 组件图     | 展示系统物理模块（前端、后端、数据库）及依赖关系                           | Docker Compose 三容器架构                          |

|     |             |                    |
|-----|-------------|--------------------|
| 部署图 | 映射软件组件到硬件节点 | 部署环境说明 (Docker 编排) |
|-----|-------------|--------------------|

### 1.3 UML 建模实践体会

通过本次项目实践，我们深刻认识到 UML 并非“画图负担”，而是沟通和验证需求的强有力工具。例如：

(1) 状态机图帮助我们在编码前识别出文章状态转换的非法路径（如已删除文章不可置顶），直接转化为 OCL 约束和代码校验逻辑。

(2) 类图中的关联基数（如 User 1 : N Post）指导了外键设计和级联策略。

(3) 用例图的“包含”和“扩展”关系清晰划分了核心功能与可选功能，支撑了 MVP 决策。

不足在于：部分细节（如异步 SQLAlchemy 交互）难以用标准 UML 表达，我们通过时序图补充说明。总体而言，UML 在需求分析和体系结构设计阶段起到了关键的知识传递和决策支撑作用。

## 第二部分：逻辑在计算机科学中的应用

### 2.1 逻辑学基础回顾

逻辑是研究推理有效性的学科，在计算机科学中主要使用**命题逻辑**和**一阶谓词逻辑**。命题逻辑以原子命题和逻辑连接词（ $\neg$ ,  $\wedge$ ,  $\vee$ ,  $\rightarrow$ ,  $\leftrightarrow$ ）构成公式，通过真值表判定永真性；一阶逻辑增加量词（ $\forall$ ,  $\exists$ ）和谓词，能够表达“所有用户都有密码”这类带变量的陈述。逻辑的核心应用包括：形式验证、程序正确性证明、数据库查询（关系代数等价）、知识表示（Prolog）、逻辑编程、约束求解等。

### 2.2 逻辑在 BlogHub 项目中的具体应用

本项目中逻辑形式化方法主要体现在以下方面：

#### 2.2.1 对象约束语言 (OCL)

OCL 是基于—阶逻辑的表达式语言，用于精确描述 UML 模型上的不变式、前置/后置条件。例如：

1.文章置顶的前提：文章必须是已发布状态

context Post inv:

self.is\_pinned = true implies self.status = PostStatus::published

2.用户不能举报自己

context Report inv:

`self.reporter_id <> self.target_type = ReportTargetType::user` implies `self.target_id`

这些 OCL 约束直接映射为后端 Pydantic 校验和数据库触发器逻辑。

### 2.2.2 状态转换的逻辑约束

板块申请状态机 (pending  $\rightarrow$  approved/rejected) 的逻辑形式化:

$\forall s: \text{SectionApplication}, \forall t: \text{Transition}$

$(\text{status}(s) = \text{pending} \wedge t = \text{approve}) \rightarrow \text{status}'(s) = \text{approved}$

该约束被编码到审批 API 的权限校验和状态更新逻辑中。

### 2.2.3 权限推理

RBAC 权限模型可用一阶逻辑描述:

$\forall u: \text{User}, \forall p: \text{Post}, \forall op: \text{Operation}$

$(\text{role}(u) = \text{admin}) \vee (\text{role}(u) = \text{moderator} \wedge \text{belongsTo}(u, p.\text{section}))$

$\rightarrow \text{permitted}(u, op, p)$

这对应代码中的 `can_manage_post(user, post)` 函数。

### 2.2.4 数据库完整性约束

SQL 中的 UNIQUE、FOREIGN KEY、CHECK 约束本质上是逻辑断言。例如:

`ALTER TABLE post_interactions ADD CONSTRAINT unique_user_post_type`

`UNIQUE (user_id, post_id, type);`

保证“同一用户对同一文章的同一互动类型至多一条记录”。

## 2.3 逻辑应用的收益

(1) 减少歧义: 自然语言需求 (“只有作者能删文章”) 被 OCL 明确为 `author_id = current_user_id`。

(2) 自动化验证: 部分 OCL 约束可通过工具 (如 USE 系统) 自动检测一致性。

(3) 测试用例生成: 逻辑约束的正反例直接导出边界测试 (如尝试置顶已删除文章应返回 400)。

## 第三部分: 软件体系结构设计 (SAD) 初稿

### 3.1 SAD 标准对比: GB/T 8567 与 IEEE 42010

| 对比维度  | GB/T 8567-2006 《计算机软件产品开发文件编制指南》 | IEEE 42010-2011 《系统和软件架构描述》                            |
|-------|----------------------------------|--------------------------------------------------------|
| 核心思想  | 文档导向，按阶段输出固定章节（引言、总体设计、接口设计等）    | 利益相关者导向，强调架构视角与关注点分离                                   |
| 架构视图  | 未内置视图框架，仅要求模块分解图、数据流图            | 强制定义架构描述框架，包含多个视图（如逻辑视图、进程视图、物理视图、开发视图、场景视图——即“4+1”模型） |
| 利益相关者 | 简要提及用户和维护人员                      | 明确要求识别利益相关者及其关注点，每个视图针对特定关注点                           |
| 表达方式  | 倾向于自然语言 + 流程图                    | 鼓励使用多视图建模（UML、SysML 等）                                 |
| 可追踪性  | 需求→设计→实现的追溯表可选                   | 每个架构决策必须关联到所满足的需求                                      |
| 适用场景  | 国内政府、军工等传统瀑布项目                   | 国际通用的现代架构描述标准，适合敏捷和复杂系统                                |

**本项目选择：**以 IEEE 42010 的“4+1 视图模型”为主框架组织 SAD，同时参考 GB/T 8567 的接口设计和运行环境章节作为补充，以兼顾规范性与国际通适性。

3.2 BlogHub 系统架构设计（4+1 视图）

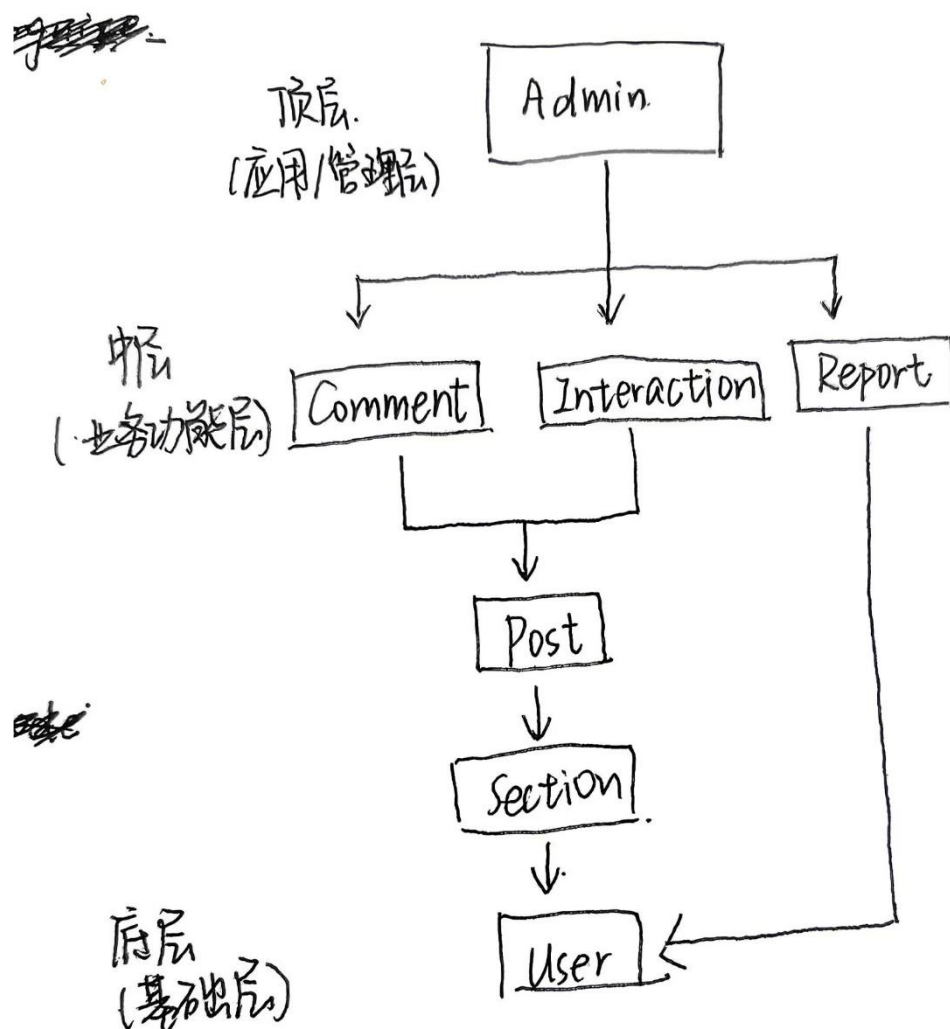
3.2.1 逻辑视图（Logical View）

逻辑视图描述系统的功能分解和模块协作，使用 UML 类图与组件图表达。

**核心模块：**

- (1) User Module：注册、登录、JWT 认证、资料管理、角色权限
- (2) Section Module：板块树管理、板块申请审批
- (3) Post Module：文章 CRUD、Markdown 渲染、置顶/隐藏、浏览计数
- (4) Comment Module：嵌套评论管理、删除权限控制
- (5) Interaction Module：点赞/收藏的幂等管理
- (6) Report Module：举报提交与处理
- (7) Admin Module：审计日志、全局管理面板

## 依赖关系



### 3.2.2 进程视图 (Process View)

本系统为 Web 应用，进程视图描述运行时进程/线程结构：

- (1) 前端进程：React SPA 运行于浏览器，单线程，通过 Axios 发起 HTTP 请求。
- (2) 后端进程：FastAPI 应用采用异步 ASGI 服务器 (Uvicorn)，每个请求在独立协程中处理；数据库操作使用 async SQLAlchemy。
- (3) 数据库进程：PostgreSQL 独立进程，维护连接池。
- (4) 部署进程：Docker Compose 管理三个容器 (frontend, backend, postgres)，无额外消息中间件。

### 3.2.3 物理视图 (Physical View)

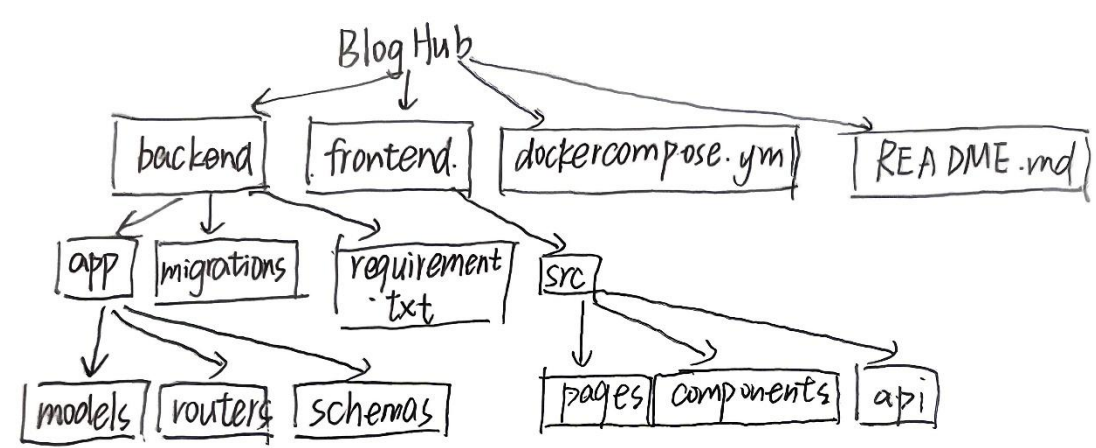
物理视图映射软件到硬件节点：

| 节点         | 软件实例                      | 资源要求                      |
|------------|---------------------------|---------------------------|
| 服务器（物理/VM） | Docker Engine             | 2 核 CPU, 4GB RAM, 20GB 磁盘 |
| 容器 1       | Nginx 代理前端静态资源 + 反向代理 API | 端口 80                     |
| 容器 2       | Uvicorn + FastAPI         | 端口 8000                   |
| 容器 3       | PostgreSQL 16             | 端口 5432, 持久化卷             |

3.2.4 开发视图（Development View）

(1) 目的：开发视图面向开发人员、技术负责人和后续维护者，描述系统的源代码组织、模块划分、技术栈依赖、构建配置和版本控制策略，确保团队高效协作和代码可维护性。

(2) 源代码组织（目录树）



(3) 模块划分与职责

| 模块   | 所属层 | 核心职责                | 主要技术                 |
|------|-----|---------------------|----------------------|
| 用户模块 | 后端  | 注册、登录、JWT、个人资料、头像上传 | passlib, python-jose |

|          |       |                             |                        |
|----------|-------|-----------------------------|------------------------|
| 板块模块     | 后端    | 板块树、申请、审批、自动晋升版主            | SQLAlchemy (自引用)       |
| 文章模块     | 后端    | CRUD、Markdown、封面、置顶/隐藏、浏览计数 | marked (后端渲染可选)        |
| 评论模块     | 后端    | 嵌套评论、删除权限                   | 递归 CTE                 |
| 互动模块     | 后端    | 点赞/收藏 (唯一约束)                | 复合唯一索引                 |
| 举报模块     | 后端    | 举报提交、处理、审计                  | 状态机                    |
| 管理后台模块   | 后端+前端 | 聚合管理、审计日志                   | Admin 路由 + 独立页面        |
| 前端 UI 模块 | 前端    | 响应式界面、路由、API 交互             | React, Tailwind, Axios |

#### (4) 构建与部署流水线

##### 开发阶段:

```
git checkout -b feature/xxx
```

→ 本地后端: `uvicorn app.main:app --reload`

→ 本地前端: `npm run dev`

→ 提交 PR → 代码审查 → 合并到 dev

##### 集成阶段:

dev 分支自动触发 (可选 CI) :

- 后端: `pytest + flake8`

- 前端: `npm run lint + npm run build`

##### 发布阶段:

```
git checkout main
```

→ `docker compose build`

→ `docker compose up -d`

→ 对外暴露端口 80 (Nginx 代理)

#### (5) 技术约束:

后端必须使用 FastAPI + SQLAlchemy 2.0 async

前端必须使用 React 18 + Vite + TailwindCSS

API 格式统一为 {code, data, msg}

3.2.5 场景视图 (Scenarios)

场景通过一组用例验证架构的完整性。典型场景“用户发表评论”的协作序列：

- (1) 前端发送 POST /api/comments, 携带 JWT
- (2) 后端 AuthMiddleware 解析用户 ID
- (3) CommentRouter 校验帖子存在、用户未禁言
- (4) CommentService.create() 写入数据库
- (5) 返回 {code:0, data: comment, msg:"ok"}
- (6) 前端更新评论列表

该场景横跨逻辑视图（模块交互）、进程视图（异步处理）和物理视图，验证了架构的端到端可行性。

3.3 架构决策记录 (ADR)

根据 IEEE 42010 要求，记录关键架构决策：

|                                        |                                     |
|----------------------------------------|-------------------------------------|
| ADR-001 选择前后端分离架构                      |                                     |
| 背景                                     | 需要支持多客户端 (Web + 后续移动端)，且团队有明确角色分工   |
| 决策                                     | 后端提供 RESTful API，前端独立开发             |
| 后果                                     | 增加 CORS 配置和接口文档成本；但获得独立部署、技术栈灵活优势   |
| ADR-002 使用 JWT 而非 Session              |                                     |
| 背景                                     | 无状态扩展性要求，前后端分离环境                    |
| 决策                                     | JWT (HS256) 存储用户角色，7 天有效期           |
| 后果                                     | 无法主动注销（可通过短有效期 + 黑名单缓解），但简化了分布式会话管理 |
| ADR-003 评论采用嵌套集 (parent_id 自引用) 而非物化路径 |                                     |
| 背景                                     | 评论深度有限 (业务建议 ≤ 3 层)，查询频率高           |
| 决策                                     | 使用 parent_id 递归查询 (在应用层或 CTE)       |



|    |                                      |
|----|--------------------------------------|
| 后果 | 深度查询有递归开销，但实现简单，适配 PostgreSQL 递归 CTE |
|----|--------------------------------------|

## 第四部分：经典软件体系结构案例研究

### 4.1 MVC (Model-View-Controller) 模式

**定义：** MVC 将应用划分为数据模型 (Model)、用户界面 (View) 和控制逻辑 (Controller)。Model 通知 View 变化，View 接收用户输入并转发给 Controller，Controller 更新 Model。

**经典案例：** Ruby on Rails、Spring MVC、Django (类 MVC)。

**与 BlogHub 的关系：** 前端 React 可以视为 View；后端 FastAPI 路由充当 Controller；SQLAlchemy Model 是 Model。但本项目未使用前端 MVC 框架，而是采用组件 + 状态管理 (类似 MVVM)。MVC 的思想仍体现在前后端职责分离上。

### 4.2 分层架构 (Layered Architecture)

**定义：** 系统自上而下分为表示层、业务逻辑层、数据访问层、数据库层。上层依赖下层，下层不感知上层。

**经典案例：** Java EE 三层架构、.NET N-Tier。

**BlogHub 应用：** 严格遵循四层——前端 (表示层) → 后端 Router (外观层) → Service (业务逻辑层) → Repository/ORM (数据访问层) → PostgreSQL。这种分层提高了测试性 (可以 mock 数据访问层) 和可维护性。

### 4.3 微服务架构 (Microservices)

**定义：** 将单一应用拆分为一组小型独立服务，每个服务围绕业务能力构建，独立部署和扩展。

**经典案例：** Netflix、Amazon、Uber。

**与 BlogHub 的对比：** 本项目为单体后端 + 独立前端，属于“前后端分离单体”。微服务对课程项目过于复杂 (分布式事务、服务发现、API 网关)，但我们在设计中预留了模块化边界 (如举报模块、审计模块可独立拆分为服务)，体现了“单体优先，微服务就绪”的思路。

### 4.4 事件驱动架构 (Event-Driven Architecture)

**定义：** 组件通过异步事件通信，事件生产者发布事件，消费者订阅处理。通常搭配消息队列 (Kafka, RabbitMQ)。

**经典案例：** 实时交易系统、IoT 数据处理。

**本项目的潜在应用：** 审计日志、通知系统可以采用事件驱动解耦。例如，文章被删除时发布 PostDeletedEvent，审计模块和通知模块各自消费。当前版本未实现，但设计

文档中预留了扩展点。

4.5 案例对比与借鉴

| 模式   | 适用场景          | 本项目借鉴点           |
|------|---------------|------------------|
| MVC  | GUI 应用、Web 应用 | 前后端分离，但未强制前端 MVC |
| 分层   | 企业级应用         | 核心架构，后端严格分层      |
| 微服务  | 大规模、多团队       | 未来扩展方向，目前单体边界清晰  |
| 事件驱动 | 异步、解耦需求       | 可用于通知和日志，当前未实现   |

通过学习这些经典案例，我们认识到没有“最好”的架构，只有“最合适”的架构。BlogHub 基于对开发周期、团队规模和交付质量的权衡，选择了**分层单体 + 前后端分离**的混合风格，既保证工程规范性，又控制了复杂度。

第五部分：软件需求规格说明（SRS）完成情况

根据实验九要求，“完成软件需求规格说明 SRS”。本小组已在实验八中完成了完整的 SRS，文档包含以下章节：

- 1. 引言（目的、范围、定义）
- 2. 功能需求（27 项，覆盖用户、板块、文章、评论、互动、举报、管理后台、文件上传）
- 3. 非功能需求（响应时间、并发、安全性、可用性、可维护性、数据一致性）
- 4. 系统约束（认证方式、数据库、API 格式、部署方式）
- 5. 数据库安全策略（密码哈希、完整性约束）
- 6. 接口规范（统一响应格式）
- 7. 角色权限矩阵

SRS 文件已按照实验八要求提交，并在此实验九报告中作为成果之一。详细内容请参见《实验八 软件需求规格说明 SRS.pdf》和《软件需求规格说明书-博客平台项目.pdf》。

第六部分：实验总结

6.1 实验总结

本次实验覆盖了 UML 建模语言学习、逻辑在计算机科学中的应用、软件体系结构设计（SAD）标准对比与撰写、经典体系结构案例研究以及 SRS 的最终完成。通过实验，我们：掌握了 UML 14 种图中最核心的 6 种（用例图、类图、状态机图、活动

图、组件图、部署图)的实际应用技巧。理解了逻辑(命题逻辑、一阶逻辑、OCL)如何从形式化层面提升需求精度和可验证性。对比了 GB/T 8567 与 IEEE 42010 的差异,并为本项目选择了合适的 SAD 编写策略。研究了 4 种经典架构模式,加深了对架构权衡的理解。完成了 BlogHub 项目的完整 SRS,并通过需求追踪矩阵确保每个需求可验证。